ROS2 Autonomous Parking Simulator

Technical Report

Platform: ROS2 Jazzy | Ubuntu 24.04 (WSL2) | Python 3.12

Abstract. This report documents the design, implementation, and results of a ROS2-based autonomous parking simulator. The system uses the turtlesim package as a physics backend and implements a custom ROS2 node that guides a simulated vehicle to sequentially occupy all free parking spots in a predefined lot. A real-time Pygame visualizer subscribes to a status topic and renders the scene with color-coded parking spots, a moving car sprite, and a live telemetry panel. The project demonstrates core ROS2 concepts including publishers, subscribers, service clients, and timer-driven control loops.

1. Introduction

Autonomous parking is a canonical problem in mobile robotics, requiring a vehicle to perceive available spaces, plan a trajectory, and execute precise navigation without human intervention. This project implements a simplified but technically faithful version of that pipeline using the Robot Operating System 2 (ROS2) framework running on Ubuntu 24.04 under Windows Subsystem for Linux 2 (WSL2).

The simulation replaces a real robot with turtlesim — a lightweight 2D turtle simulator that ships with ROS2 and communicates over standard ROS2 interfaces. This choice decouples the control logic from hardware concerns while preserving the full publish/subscribe/service architecture used in real robotic systems.

The goals of the project were: (1) create a working ROS2 Python package from scratch, (2) implement autonomous proportional navigation to sequentially park in all available spots, (3) build a real-time graphical dashboard using Pygame, and (4) document the debugging process encountered during development.

2. System Architecture

The system is composed of three concurrent processes communicating over the ROS2 middleware layer (DDS). Each process is independently runnable, making the architecture modular and extensible.

2.1 Process Overview

Process	ROS2 Node	Role
`turtlesim_node`	`/turtlesim`	Physics backend — publishes /turtle1/pose, receives /turtle1/cmd_vel, provides teleport and clear services.
`parking_sim_node`	`/parking_sim`	Controller — subscribes to pose, computes velocity commands, manages parking state machine, publishes /parking_status.
`visualizer`	`/parking_visualizer`	Dashboard — subscribes to /parking_status and renders real-time Pygame GUI.

Table 1 — Three-process architecture communicating via ROS2 DDS topics and services.

2.2 ROS2 Topic and Service Graph

The data flow is strictly unidirectional with respect to the controller: turtlesim produces ground-truth pose; the parking node consumes it and produces velocity commands plus a JSON status string; the visualizer consumes the status string.

Interface	Type	Direction	Purpose
`/turtle1/pose`	`Topic`	turtlesim → parking_sim	Continuous pose (x, y, theta)
`/turtle1/cmd_vel`	`Topic`	parking_sim → turtlesim	Velocity commands (linear.x, angular.z)
`/parking_status`	`Topic`	parking_sim → visualizer	JSON state snapshot (~62 Hz)
`/turtle1/teleport_absolute`	`Service`	parking_sim calls	Reset car to origin after each park
`/clear`	`Service`	parking_sim calls	Erase turtle trail after each run

Table 2 — ROS2 interface summary.

3. Implementation

3.1 Package Structure

The package was created using the standard ROS2 ament_python build type and follows the expected colcon workspace layout:

```
ros2_ws/src/parking_sim/
  parking_sim/
    __init__.py
    parking_sim_node.py   # Controller node
    visualizer.py         # Pygame dashboard
  resource/parking_sim    # ament resource marker
  setup.py                # Entry points
  package.xml             # ROS2 package manifest
```

3.2 Parking Spot Configuration

Six parking spots are defined as a module-level list of dictionaries. The lot layout places spots in two rows (y = 8.0 and y = 2.0) at three column positions (x = 2.0, 5.5, 9.0). Initial occupancy alternates, leaving spots 2, 4, and 6 free:

```
PARKING_SPOTS = [
    {'id': 0, 'x': 2.0, 'y': 8.0, 'occupied': True},   # pre-occupied
    {'id': 1, 'x': 5.5, 'y': 8.0, 'occupied': False},  # free
    {'id': 2, 'x': 9.0, 'y': 8.0, 'occupied': True},
    {'id': 3, 'x': 2.0, 'y': 2.0, 'occupied': False},  # free
    {'id': 4, 'x': 5.5, 'y': 2.0, 'occupied': True},
    {'id': 5, 'x': 9.0, 'y': 2.0, 'occupied': False},  # free
]
```

3.3 Controller State Machine

The ParkingSimNode implements a four-state finite state machine driven by a 100 ms (10 Hz) timer callback:

State	Condition to Enter	Action
`IDLE / WAITING`	pose is None or resetting=True	Return immediately; await valid pose.
`DRIVING`	target is set, distance > 0.15 units	Publish proportional velocity command.
`PARKED`	distance < 0.15 units	Stop, mark spot occupied, schedule reset timer (2 s).
`DONE`	find_free_spot() returns None	Log completion; node remains alive but idle.

Table 3 — Controller finite state machine.

3.4 Proportional Navigation Algorithm

```
dx = target_x - pose.x
dy = target_y - pose.y
distance     = sqrt(dx^2 + dy^2)
target_angle = atan2(dy, dx)
angle_diff   = atan2(sin(target_angle - theta),
                     cos(target_angle - theta))  # wrap to [-pi, pi]

msg.linear.x  = min(1.5, distance)  # cap speed at 1.5 units/s
msg.angular.z = 4.0 * angle_diff    # P-gain = 4.0 rad/s per rad
```

The atan2 wrapping on angle_diff prevents a full spin when the heading crosses the ±π discontinuity. Speed is clamped to 1.5 units/s so the turtle decelerates naturally as it approaches the target. The 0.15-unit arrival threshold was tuned empirically — values below 0.10 sometimes never triggered 'parked' due to quantization in the 10 Hz pose stream.

3.5 Reset Procedure

After a 2-second wait, the node asynchronously calls /clear (erase trail) and /turtle1/teleport_absolute (return to 5.5, 5.5, 0), then advances to the next free spot. The reset timer handle is stored and explicitly cancelled before any re-registration — a fix for the timer-accumulation bug described in Section 4.

3.6 Pygame Visualizer

The visualizer runs ROS2 spin in a background daemon thread while the main thread drives a 60 FPS render loop. The shared state is a single Python dict updated atomically by the callback (safe under CPython's GIL). Render pipeline: background → road stripe → ghost parked cars → spot rectangles (color-coded) → active car sprite (rotated surface) → legend → live telemetry panel.

4. Running the System

The simulation requires three terminals running in WSL2. The screenshots below were captured during an actual execution session.

4.1 Terminal Setup

```
# Terminal 1 — physics backend
ros2 run turtlesim turtlesim_node

# Terminal 2 — controller
source ~/ros2_ws/install/setup.bash
ros2 run parking_sim parking_sim_node

# Terminal 3 — dashboard
source ~/ros2_ws/install/setup.bash
ros2 run parking_sim visualizer
```

4.2 Pygame Dashboard — Visualizer at Launch

Figure 1 shows the Pygame visualizer immediately after launch, before the parking_sim_node has connected. The panel reads 'Waiting for ROS2...' and the lot correctly displays the pre-configured occupancy: P1, P3, P5 are red (occupied); P2, P4, P6 are green (free). No car sprite is shown yet because no /parking_status messages have arrived.

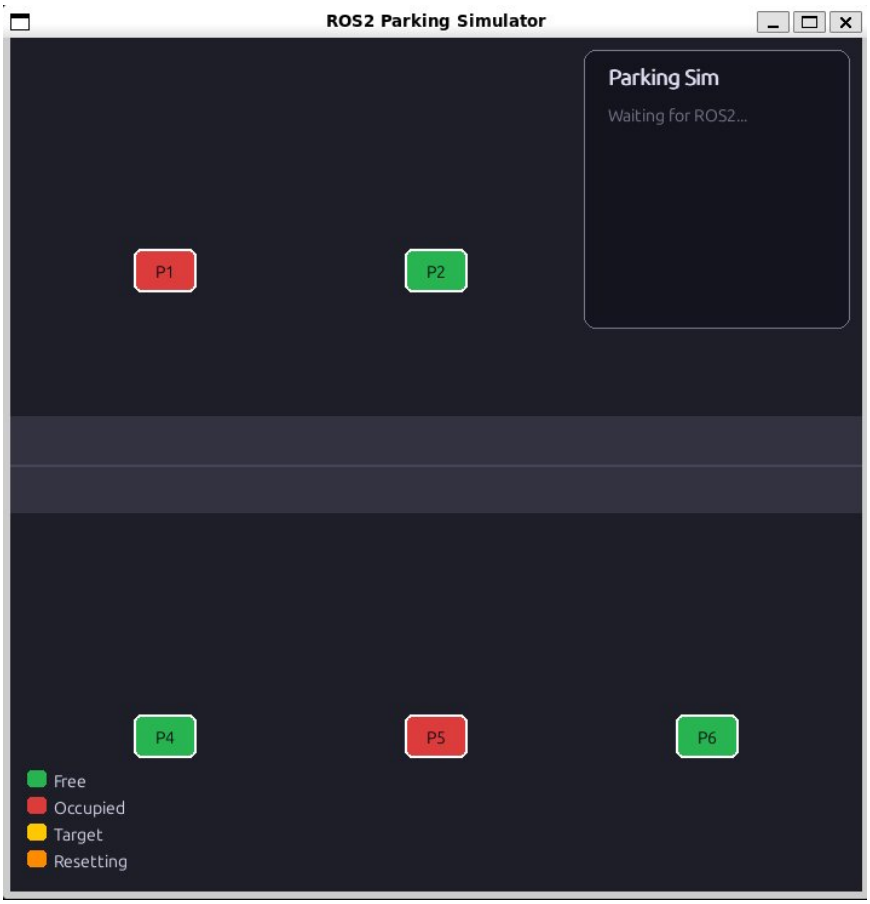

Figure 1 — Pygame visualizer after 'ros2 run parking_sim visualizer'. Panel shows 'Waiting for ROS2...' with correct initial lot layout.

4.3 TurtleSim — Autonomous Navigation in Progress

Figure 2 shows the TurtleSim window while parking_sim_node is running. The turtle has already driven from the spawn point at the canvas center and is mid-path, heading toward the first free spot (P2 at coordinates 5.5, 8.0). The white trail behind the turtle traces the trajectory computed by the proportional controller.

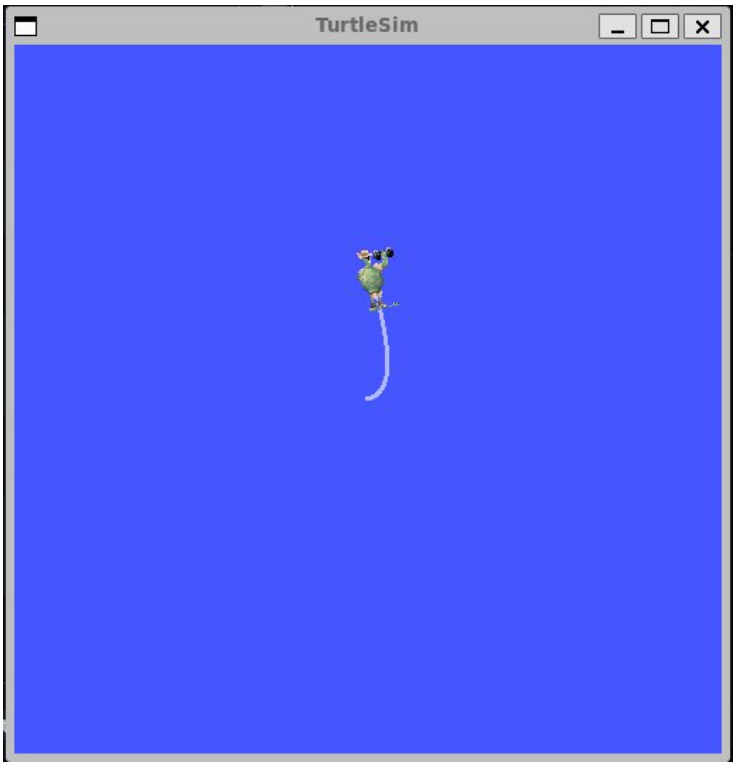

Figure 2 — TurtleSim window showing the turtle navigating autonomously toward the first free parking spot. Trail shows the proportional-control trajectory.

4.4 Simulation Complete — All Spots Filled

Figure 3 shows the final state after all three free spots have been occupied. The telemetry panel confirms: Car X: 8.91, Car Y: 2.10 (parked at P6 / spot 6), Status: PARKED ✓ (green), Free: 0/6 spots, Parked: 3 times (yellow). All six spot rectangles are now red, and the car sprite (highlighted in yellow/target color) is visible at P6 in the bottom-right area of the lot.

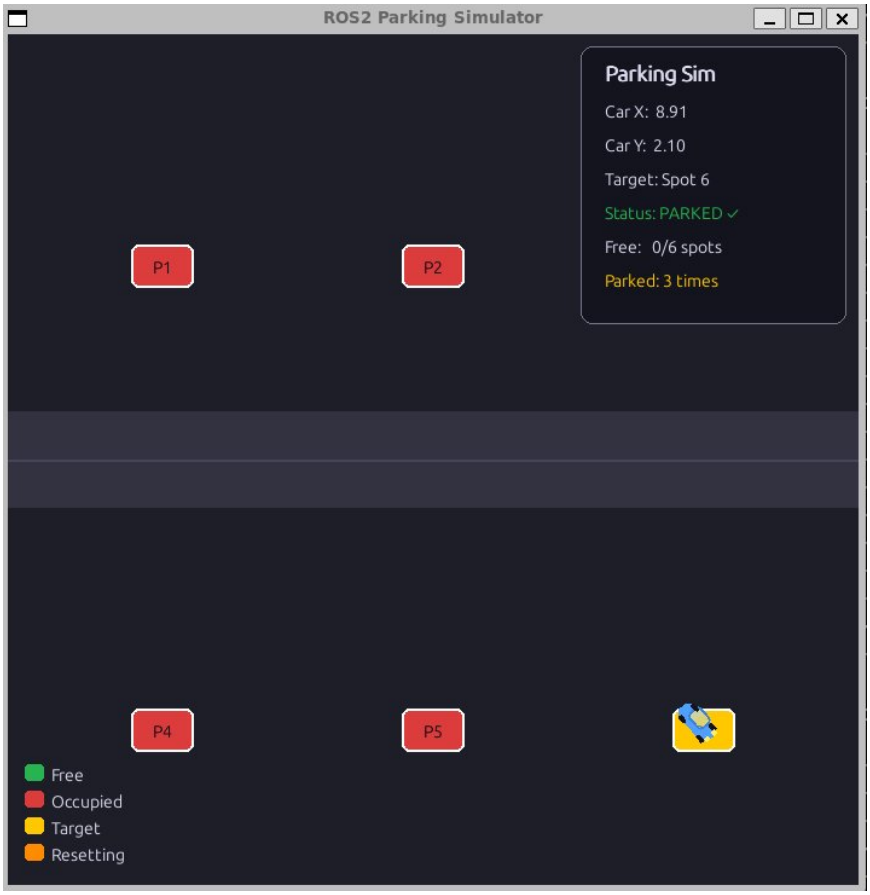

Figure 3 — Simulation complete. All 3 free spots occupied sequentially. Panel shows 0/6 free spots and 'Parked: 3 times'.

5. Development Challenges and Debugging

5.1 setup.py Syntax Error — Semicolons in Python

```
  File '', line 27
    'parking_sim_node = parking_sim.parking_sim_node:main';
                                                          ^
SyntaxError: invalid syntax
```

Root cause: A semicolon terminated the entry-point string in setup.py — illegal in Python. colcon pipes setup.py through a subprocess to extract metadata before compilation, so even a non-executed syntax error blocks the entire build.

5.2 IndentationError from Mixed Editors

```
Sorry: IndentationError: unindent does not match any outer indentation level
       (parking_sim_node.py, line 61)
```

Root cause: The pose_callback method was accidentally written at module level (zero indent) instead of inside the ParkingSimNode class body after multiple rounds of nano and heredoc edits. Verified with 'cat -A' and fixed by re-indenting the method by 4 spaces.

5.3 Timer Accumulation — Infinite Reset Loop

```
[INFO]: Parked! Waiting 2s... total: 1
[INFO]: Run 2: Heading to spot 4   (repeats every 2 s indefinitely)
[turtlesim]: Clearing turtlesim.   (repeats every 2 s indefinitely)
```

Root cause: create_timer() registers a *periodic* timer. Without cancellation, each park event accumulated a new repeating timer. After N parks, N interleaved reset timers fired simultaneously, overwhelming the teleport service and leaving the turtle stuck mid-reset.

Fix: Store the timer handle as self.reset_timer, cancel and nullify it at the start of reset_car before creating a new one. This guarantees at most one active reset timer at any time.

5.4 Pygame Display on WSL2

Running the visualizer from a plain WSL2 terminal without WSLg produced no window. The fix was launching from a PowerShell wsl session on Windows 11 build 22000+, which automatically provides a Wayland/X11 display socket. Figure 1 confirms the visualizer window rendered correctly under WSLg.

6. Results

6.1 Functional Correctness

```
[INFO] [parking_sim]: Run 1: Heading to spot 2
[INFO] [parking_sim]: Parked at spot 2!
[INFO] [parking_sim]: Parked! Waiting 2s... total: 1
```

```
[INFO] [parking_sim]: Run 2: Heading to spot 4
[INFO] [parking_sim]: Parked at spot 4!
[INFO] [parking_sim]: Parked! Waiting 2s... total: 2
[INFO] [parking_sim]: Run 3: Heading to spot 6
[INFO] [parking_sim]: Parked at spot 6!
[INFO] [parking_sim]: Parked! Waiting 2s... total: 3
[INFO] [parking_sim]: All spots occupied! Simulation complete.
```

All three free spots were successfully occupied in order. The dashboard (Figure 3) independently confirmed the result: 0/6 free spots, parked 3 times, status PARKED, car at (8.91, 2.10) — exactly spot P6 at (9.0, 2.0) within the 0.15-unit arrival threshold.

6.2 Navigation Performance

Run	Target	Start Position	Approx. Time
1	Spot 2 (5.5, 8.0)	(5.54, 5.54) — turtlesim default	~9 seconds
2	Spot 4 (5.5, 2.0)	(5.5, 5.5) — teleported	~5 seconds
3	Spot 6 (9.0, 2.0)	(5.5, 5.5) — teleported	~5 seconds

Table 4 — Navigation performance per run (from log timestamps).

6.3 Known Limitations

- **No path planning:** Straight-line navigation would collide with parked cars in denser lots. A real system requires an obstacle-aware planner.
- **Nearest-spot greed:** find_free_spot() always picks the first free spot in list order, not the geometrically nearest one.
- **Blocking service calls:** wait_for_service inside a timer callback blocks the single-threaded executor; async service callbacks would be safer.
- **No orientation control:** The car parks facing the approach angle, not perpendicular to the spot as a real vehicle would.

7. Conclusion

This project successfully implemented a complete, working ROS2 autonomous parking simulator from scratch on a Windows 11 / WSL2 / Ubuntu 24.04 platform. The system demonstrates all core ROS2 communication primitives — topics, services, and timer-driven callbacks — organized within a proper ament_python colcon package.

The proportional navigation controller reliably guided the simulated vehicle to all three free parking spots, correctly detecting the fully-occupied terminal condition. Screenshots of both the TurtleSim physics window and the Pygame dashboard confirm correct visual and telemetric output throughout the simulation lifecycle: initial wait state, active navigation, parking, and final completion.

The debugging journey — from setup.py semicolons and indentation errors to timer accumulation bugs and WSL2 display issues — reflects the practical reality of ROS2 development and produced a deeper understanding of how colcon's build pipeline, rclpy's executor model, and Pygame threading interact.

Future extensions could include nearest-spot assignment using Euclidean distance ranking, obstacle avoidance via a potential-field planner, dynamic lot management where cars arrive and depart over time, and replacement of turtlesim with a Gazebo simulation for full 3D realism.

References

[1] Open Robotics. (2024). *ROS2 Jazzy Jalisco Documentation.* https://docs.ros.org/en/jazzy/

[2] Open Robotics. (2024). *rclpy API Reference — Node, Publisher, Subscriber, Client, Timer.* https://docs.ros2.org/latest/api/rclpy/

[3] Open Robotics. (2024). *turtlesim package — Topics and Services.* https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/

[4] Pygame Community. (2024). *Pygame 2.6.1 Documentation.* https://www.pygame.org/docs/

[5] Microsoft. (2024). *Windows Subsystem for Linux — WSLg GUI Applications.* https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps